S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 05

Aim: Write a program to implement Shortest Job First (SJF) Preemptive Scheduling for three processes and calculate the total context switches and average waiting time. The processes have burst times 10ns, 20ns, and 30ns, arriving at 0ns, 2ns, and 6ns, respectively.

Name: Ayushi Manoj Kukde

USN: EE24TCD001

Semester / Year: 4th

Sem/ 2nd Year

Academic Session:

2025 -2026

Date of Performance:

Date of Submission:

❖ **Aim:** Write a program to implement Shortest Job First (SJF) Preemptive Scheduling for three processes and calculate the total context switches and average waiting time. The processes have burst times 10ns, 20ns, and 30ns, arriving at 0ns, 2ns, and 6ns, respectively.

❖ **Objectives:**

Understand SJF Preemptive Scheduling: Implement the **Shortest Job First (SJF) Preemptive Scheduling** algorithm to manage CPU execution efficiently.
Calculate Context Switches: Determine the total number of context switches required for the given set of processes.
Evaluate Waiting Time: Compute the **average waiting time** for all processes before getting CPU execution.

❖ **Requirements:**

✔ **Hardware Requirements:**

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

✔ **Software Requirements:**

- Operating System: Linux/Unix-based
- Shell: Bash 4.0 or higher
- Text Editor: Nano, Vim, or any preferred editor

❖ **Theory:**

CPU Scheduling in Operating Systems

Introduction

Scheduling is the method by which processes are given access to the CPU. Efficient scheduling is essential for optimal system performance and user experience. There are two primary types of CPU scheduling: **Preemptive Scheduling** and **Non-Preemptive Scheduling**.

Understanding the differences between these scheduling types helps in designing and choosing the right scheduling algorithms for different operating system

1. Preemptive Scheduling

In **Preemptive Scheduling**, the operating system can interrupt or preempt a running process to allocate CPU time to another process, typically based on priority or time-sharing policies. A process can be switched from the **running state to the ready state** at any time.

Algorithms Based on Preemptive Scheduling:

a. **Round Robin (RR)**

b. **Shortest Remaining Time First (SRTF)**

c. **Priority Scheduling (Preemptive version) Example:**

In the following case, **P2 is preempted at time 1** due to the arrival of a higher-priority process.

Advantages of Preemptive Scheduling:

✔ Prevents a process from monopolizing the CPU, improving system reliability.

✔ Enhances **average response time**, making it beneficial for multi-programming environments.

✔ Used in modern operating systems like **Windows, Linux, and macOS**.

Disadvantages of Preemptive Scheduling:

✗ More complex to implement.

✗ Involves **overhead** for suspending a running process and switching contexts.

✗ **May cause starvation** if low-priority processes are frequently preempted.

✗ Can create **concurrency issues**, especially when accessing shared resources.

---

2. **Non-Preemptive Scheduling**

In **Non-Preemptive Scheduling**, a running process cannot be interrupted by the operating system. It continues executing until it **terminates** or **enters a waiting state** voluntarily.

Algorithms Based on Non-Preemptive Scheduling:

a. **First Come First Serve (FCFS)**

b. **Shortest Job First (SJF - Non-Preemptive)**

c. **Priority Scheduling (Non-Preemptive version) Example:**

Below is a **Gantt Chart** based on the **FCFS algorithm**, where each process executes fully before the next one starts.

Advantages of Non-Preemptive Scheduling:

✔ **Easy to implement** in an operating system (used in older versions like Windows 3.11 and early macOS).

✔ **Minimal scheduling overhead** due to fewer context switches.

✔ **Less computational resource usage**, making it more efficient for simpler systems.

Disadvantages of Non-Preemptive Scheduling:

✗ **Risk of Denial of Service (DoS) attacks**, as a process can monopolize the CPU.

✗ **Poor response time**, especially in multi-user systems.

3. **Differences Between Preemptive and Non-Preemptive Scheduling**

Parameter	Preemptive Scheduling	Non-Preemptive Scheduling
Basic Concept	CPU time is allocated for a **limited time**.	CPU is held until process **terminates** or enters waiting state.
Interrupts	Process **can be interrupted**.	Process **cannot be interrupted**.
Starvation	Frequent high-priority processes may starve low-priority ones.	A long-running process can starve later-arriving shorter processes.
Overhead	Higher overhead due to frequent **context switching**.	Minimal overhead.
Flexibility	More flexible (critical processes get priority).	Rigid scheduling approach.
Response Time	Faster response time.	Slower response time.
Process Control	**OS has more control** over scheduling.	OS has **less control** over scheduling.
Concurrency Issues	**Higher**, as processes may be preempted during shared resource access.	**Lower**, as processes run to completion.
Examples	Round Robin, SRTF.	FCFS, Non-Preemptive SJF.

4. Frequently Asked Questions (FAQs)

a. How is priority determined in Preemptive scheduling?

Ans: Preemptive scheduling systems assign priority based on **task importance, deadlines, or urgency**. Higher-priority tasks execute before lower-priority ones.

b. What happens in non-preemptive scheduling if a process does not yield the CPU?

Ans: If a process does not voluntarily yield the CPU, it can lead to **starvation or deadlock**, where other tasks are unable to execute.

.

c. Which scheduling method is better for real-time systems?
Ans: Preemptive scheduling is better for **real-time systems**, as it allows high- priority tasks to execute immediately.

❖ CODE

~

GNU nano 8.7 sjf_preemptive.c

```
#include <stdio.h>
#include <limits.h>

int main() {
    int n = 3;
    int arrival[] = {0, 2, 6};
    int burst[] = {10, 20, 30};
    int remaining[] = {10, 20, 30};
    int waiting[3] = {0, 0, 0};

    int time = 0, completed = 0;
    int prev = -1;
    int context_switches = 0;

    while (completed < n) {
        int shortest = -1;
        int min_time = INT_MAX;

        for (int i = 0; i < n; i++) {
            if (arrival[i] <= time && remaining[i] > 0 && remaining[i] < min_time) {
                min_time = remaining[i];
                shortest = i;
            }
        }

        if (shortest == -1) {
            time++;
            continue;
        }

        if (prev != shortest && prev != -1) {
            context_switches++;
        }

        remaining[shortest]--;
        time++;

        if (remaining[shortest] == 0) {
            completed++;
            waiting[shortest] = time - arrival[shortest] - burst[shortest];
        }

        prev = shortest;
    }

    float avg_wait = 0;
    for (int i = 0; i < n; i++) {
        avg_wait += waiting[i];
    }
    avg_wait /= n;

    printf("Total Context Switches = %d\n", context_switches);
    printf("Average Waiting Time = %.2f ns\n", avg_wait);

    return 0;
}
```

[Read 56 lines]

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket M-B Previous ^← Back ^→ Prev Word
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^B Where Was M-F Next ^ Forward ^↓ Next Word

❖ Output:

~

```
Dell@DESKTOP-D7MT61A MSYS ~
$ nano sjf_preemptive.c

Dell@DESKTOP-D7MT61A MSYS ~
$ gcc sjf_preemptive.c -o sjf

Dell@DESKTOP-D7MT61A MSYS ~
$ ./sjf
Total Context Switches = 2
Average Waiting Time = 10.67 ns

Dell@DESKTOP-D7MT61A MSYS ~
$
```

Conclusion: Preemptive scheduling offers better responsiveness but adds complexity, while non-preemptive scheduling is simpler but may cause inefficiencies. The choice depends on system needs, with preemptive suited for multitasking and non-preemptive for low-overhead scenarios.